AccDRC: FPGA Acceleration for VLSI DRC with Cell-aware Layout Partitioning

Ruping Zhou[1], Zexu Zhang[1], You Hu[2], Jiahao Wang[2], Haodong Lu[2], Genggeng Liu[1,*], Jianli Chen[2], Kun Wang[2,*]
[1]Fuzhou University, Fuzhou, China [2]Fudan University, Shanghai, China

Abstract

Design Rule Checking (DRC) is a critical yet computation-intensive stage in modern very large scale integration design. As processes evolve, the high computational cost of DRC has become a significant bottleneck for design efficiency. Current acceleration approaches do not fully leverage the parallelism of DRC, resulting in limited performance gains. To address this challenge, we propose *AccDRC*, an FPGA-accelerated DRC based on software–hardware co-design. On the software, we design a cell-aware partitioning strategy with a data preparation and task encapsulation mechanism, which reorganize layouts into balanced task units tailored for FPGA processing. On the hardware, we implements an acceleration architecture consisting of a locality-preserving data-loading module, a unified and reconfigurable check core, and a sparse result write-back module. This architecture exploits DRC's locality, structural commonality across rules, and sparse violation outcomes, enabling high-throughput dataflow execution with multi-level parallelism. Experimental results show that *AccDRC* achieves 522.11×~1071.03× speedup over the CPU-based DRC tool KLayout, and 9.62×~26.07× speedup over the state-of-the-art GPU-based DRC tool OpenDRC.

1 Introduction

Design Rule Checking (DRC) is a crucial step in physical design and verification of Very Large Scale Integration (VLSI), ensuring that the layout complies with the geometric and connectivity constraints required by manufacturing processes, as shown in Fig. 1(a). It directly impacts the manufacturability, yield, and reliability of the chip. With advanced process nodes, the computational cost of DRC has increased sharply, making layout flattening and checking computationally expensive, as shown in Fig. 1(b). Additionally, since DRC is invoked repeatedly during the design process, its runtime has become a major bottleneck that constrains the design cycle. This expensive computational burden underscores the need for efficient DRC acceleration approaches.

Current major DRC engines, such as KLayout and Calibre use multi-core CPUs to accelerate DRC. Yet the limited number of CPU cores, together with the overhead of thread scheduling and synchronization, limits the performance gains of CPU-based parallelism. To address this, [10, 11] adopt distributed computing architectures to enhance DRC efficiency through multi-node collaboration. However, in large-scale designs, distributed computing faces challenges including complex task partitioning, maintaining global consistency, and balancing the workload across nodes.

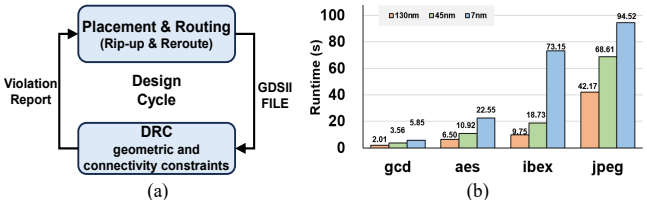


Figure 1: Illustraion of DRC in VLSI design. (a) DRC is repeatedly invoked throughout the design cycle. (b) DRC runtime grows substantially as process technology scales down to advanced nodes.

GPUs, known for their massive parallel computing capabilities, have become effective tools for parallelizing DRC. [4, 6, 13] restructure the Boolean union and sweepline operation kernels of traditional DRC to adapt to GPU architectures. [5] leverages CPU and GPU resources to enhance efficiency. [7] extends the GPU-accelerated DRC to handle complex scenarios, including non-Manhattan geometries and advanced packaging processes. Nevertheless, frequent kernel launches undermine the benefits of GPU parallelization, especially for fine-grained tasks. Moreover, irregular data accesses and conditional branch divergences in DRC can lead to thread serialization, limiting GPU throughput and overall computational capacity.

FPGAs offer distinct advantages for DRC acceleration owing to their programmable hardware characteristics. They can implement customized check modules as resident hardware pipelines, enabling multi-grained parallelism without incurring frequent kernel launch overhead. Additionally, FPGAs allow designers to explicitly control data paths, including bit-width optimization and memory access patterns, ensuring deterministic throughput and stable performance. Early research [2, 8, 9] verifies the feasibility of FPGA-based DRC acceleration by offloading specific operations to hardware. However, these approaches fail to fully leverage the FPGAs' programmability and the DRC's parallelism, leading to limited acceleration. Additionally, they fail to effectively handle hierarchical layout data structures, limiting the scalability of their FPGA-based DRC to large-scale chip designs that commonly adopt hierarchical representations.

In this paper, we propose *AccDRC*, an FPGA-based acceleration architecture for DRC. *AccDRC* achieves substantial speedup through efficient software–hardware co-design and a cell-aware layout partitioning strategy for efficient layout processing. The main contributions are summarized as follows.

*Corresponding authors are *Genggeng Liu* and *Kun Wang*. Email: liu_genggeng@fzu.edu.com, kun.wang@ieee.org.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
DAC '26, Long Beach, CA, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804092

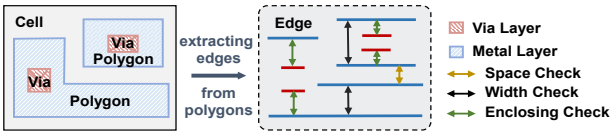


Figure 2: Extracting edges from polygons and unifying space, width, and enclosure checks into edge-based distance-rule evaluations (illustrated with horizontal edges).

- We propose a cell-aware partitioning strategy that divides the layout into bins and generates task units with high spatial locality and balanced workloads for parallel scheduling.
- We design a data preparation and task encapsulation mechanism that applies polygon flattening and two-level encapsulation to ensure data accuracy, reduce redundant computation, and support multi-level parallelism.
- We propose a reconfigurable, locality-preserving hardware architecture that leverages unified multi-rule processing, multi-level parallelism, and sparse result writeback to obtain high-throughput, scalable DRC acceleration.
- Experimental results show that our software–hardware co-design framework achieves 522.11×∼1071.03× speedup over the CPU-based DRC tool KLayout, and 9.62×∼26.07× speedup over the state-of-the-art GPU-based DRC tool OpenDRC.

2 Background and Motivation

2.1 DRC Rules

DRC focuses on checking whether geometric objects in a layout comply with manufacturing constraints, among which width, space, and enclosure constitute the most fundamental and widely applied rules. These rules share a common computational structure: they can all be transformed from polygon-level relationships into edge-based distance rules, where the distance between any two edges must not be smaller than a predefined threshold. Distance rules first require extracting the edges involved in the check and then applying a unified Euclidean distance formula to determine whether edge pairs satisfy the constraints. This uniform abstraction significantly reduces algorithmic divergence across rule types and exposes a consistent arithmetic pattern that is well suited for hardware pipelining. For instance, the polygons in Fig. 2 are transformed into edge-based distance checks corresponding to width, space, and enclosure. Such rules represent the typical computational pattern of DRC, motivating the design of a standardized processing paradigm for efficient FPGA implementation.

2.2 Layout Representations

There are primarily two organizational models for layout data: hierarchical and flattened modes. These models differ fundamentally in how they store and manage polygon objects, which directly impacts the complexity of DRC.

Hierarchical mode. The hierarchical structure of cells and their references (cell_refs) is preserved. To restore the complete layout, the hierarchy must be recursively expanded according to the reference relationships. Fig. 3(a) illustrates a hierarchical layout with two levels of reference relationships.

Flattened mode. The hierarchical reference structure is completely flattened. As depicted in Fig. 3(b), all polygons in each cell

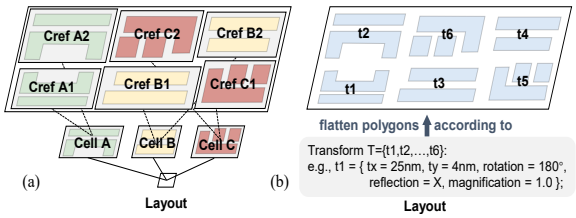


Figure 3: Comparison of hierarchical and flattened layout representations. (a) The hierarchical layout preserves the cell-reference relationships. (b) The flattened layout instantiates all polygons into a global coordinate space.

are instantiated with the transformation parameters of corresponding references and mapped into a single global coordinate system, yielding a flat collection of polygons for the entire layout.

Flattened-mode DRC is easy to implement and ensures accuracy, but incurs significant storage and computational overhead in large-scale designs. Conversely, hierarchical-mode DRC reduces the data volume, yet it is more challenging to implement and may compromise accuracy. This motivates us to propose an appropriate layout processing approach that integrates the advantages of both modes, thereby enhancing DRC efficiency while preserving accuracy.

2.3 Motivation

The DRC process imposes a heavy computational burden on modern designs and exhibits a high degree of parallelism. Most rules depend primarily on geometric relations within a local region and typically do not require inter-region or global information exchange, allowing checks in different regions to be performed simultaneously. In addition, distance calculations between edges are independent, making them well-suited for parallel processing. Moreover, DRC is characterized by massive data volume and intensive computation, further amplifying the need for efficient acceleration.

FPGAs are particularly well-suited to exploit DRC's parallelism. They can construct deeply pipelined parallel circuits and efficiently schedule many independent DRC tasks at the hardware level. With flexible hardware configuration, the degree of parallelism can be matched to different design scales. Compared to the limited parallelism of CPUs and the instability of GPUs in memory access and control branches, FPGAs utilize programmable hardware units and customizable data paths to transfer data between memory and compute units efficiently. This avoids situations where memory cannot keep up with computation or compute units remain idle, and helps eliminate bandwidth bottlenecks and load imbalance. As a result, FPGAs can provide faster and more stable acceleration while maintaining high resource utilization, making them an ideal platform for large-scale DRC.

3 Framework Overview

The *AccDRC* framework adopts a three-stage workflow: layout partitioning, data reorganization, and FPGA-accelerated checking. The first two stages, executed on the CPU, ensure efficient, accurate, and locality-aware data delivery to the FPGAFurthermore, we introduce a unified processing paradigm for distance rules to guide their hardware implementation. The third stage achieves acceleration on the FPGA. Collectively, these three stages facilitate a complete and efficient FPGA-accelerated DRC workflow, as illustrated in Fig. 4.

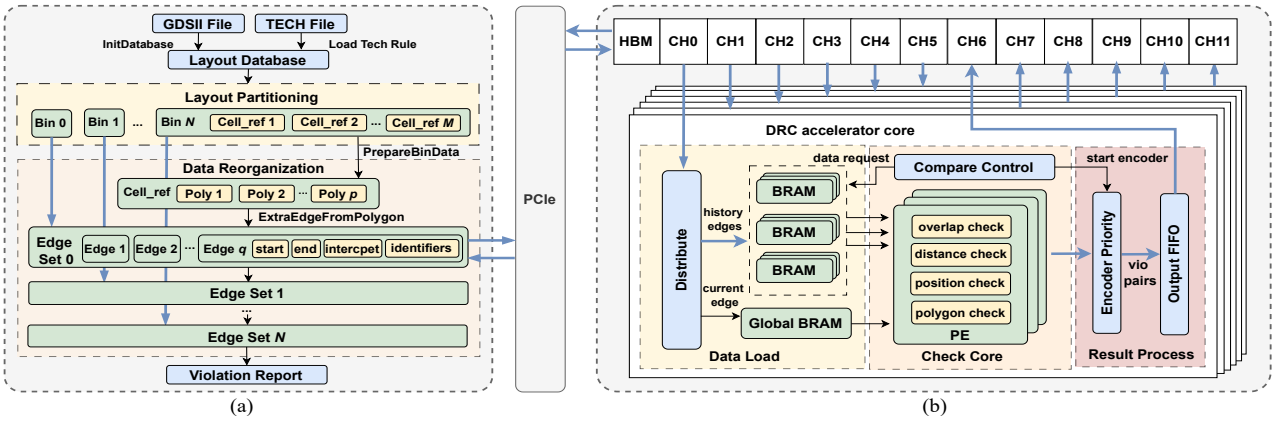


Figure 4: The framework of *AccDRC*. (a) The design flow of the software; (b) The acceleration architecture of the FPGA.

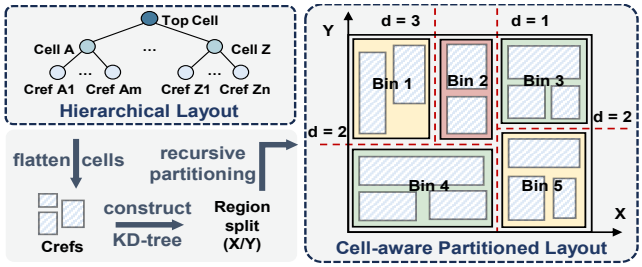


Figure 5: A cell-aware partitioned layout generated by recursively dividing each region along its longer axis (x or y), where the partition depth does not exceed 3 and the number of *cell_def* in each bin does not exceed 4.

Layout Partitioning. *AccDRC* first parses the input layout and the technology file. It then flattens cell_refs and clusters them into bins, each serving as a fundamental task unit for subsequent checks.

Data Reorganization. For each bin, *AccDRC* locally flattens polygons of cell_refs and decomposes them into orthogonal edges. The edge data from all bins is then uniformly encapsulated and transferred via PCIe to the FPGA.

FPGA-Accelerated DRC. On the FPGA, data are efficiently distributed through an optimized on-chip memory system and transmitted to multiple Processing Elements (PEs). A reconfigurable array of checkers performs parallel violation checking, after which the results are compressed and sent back to the host CPU.

4 Software Design

4.1 Layout Partitioning

We introduce a cell-aware partitioning strategy that divides the global layout into local bins, which serve as independent task units for parallel calculations.

Cell-Aware Flattening. Unlike conventional flattening, we only flatten the original cells into *cell_refs* based on hierarchical reference relationships, without flattening the polygons, thereby reducing the partitioning complexity exponentially. The center point of the *Minimum Bounding Box* (*MBB*) of each *cell_ref* is recorded as its keyword for subsequent clustering.

Recursive Partitioning. The partitioning process is illustrated in Fig. 5. For the target layer, the union of all *cell_ref MBBs* defines the global *MBB*, the root node for recursive partitioning. A KD-Tree is then constructed using the keywords of each *cell_ref*. Each node corresponds to a partitioned layout region and maintains the set of contained *cell_refs*. The partitioning rules are as follows.

- If the number of *cell_refs* in a node exceeds the threshold and the tree depth has not reached its maximum, the node is split along the longest axis of its *MBB*.
- If no further partitioning is performed, the node is marked as a leaf node, representing the final bin. Its *MBB* is then expanded by a halo threshold to preserve cross-bin geometric relationships.

The layout is organized into a tree with the global MBR as the root node and bins as the leaf nodes, each maintaining a list of the *cell_refs* it contains. This strategy rapidly decomposes the layout into balanced and independent task units while preserving local geometric integrity and laying the foundation for parallel scheduling of large-scale layouts on hardware.

4.2 Data Reorganization

We introduce a bin-level data preparation and task encapsulation mechanism, in which polygons within each bin are flattened to generate orthogonal edge data serving as the input of the task unit.

Data Preparation. We defer the flattening of polygons to this stage at the bin level, effectively reducing memory overhead . Simultaneously, the polygon data within each bin remains equivalent to those produced by global flattening, ensuring DRC accuracy. For each polygon, the edge set is constructed from the flattened point set and uniformly represented as orthogonal edge data.

Generation of Pruning Identifiers. Lightweight identifiers, e.g., key geometric attributes of edges, are attached to the generated edge data to minimize redundant computation. The identifiers enable hardware-side pruning through simple matches, avoiding complex geometric calculations. Testing demonstrates that this identifier-based pruning reduces edge pair comparisons by approximately 21% for space checks, 55% for width checks, and lowers the complexity of cross-layer edge matching from $O(N_{via} \times N_{metal})$ to near $O(N_{via})$ for enclosure checks.

Task Encapsulation. A two-level encapsulation scheme is employed to accommodate multi-level parallel computing on the FPGA. Specifically, individual edges are treated as the smallest processing units, allowing fine-grained parallelism; edges within the same

Algorithm 1 Distance-Based Checks

Require: rule $r \in \{\text{Space}, \text{Width}, \text{Enclosure}\}$, edge set P
Ensure: violation set V
1: $V \leftarrow \emptyset$
2: $\theta \leftarrow \text{GetThreshold}(r)$
3: **for** each pair $(e_1, e_2) \in \mathcal{P}$ **do**
4: **if** not $\text{PruneByRule}(r, e_1, e_2)$ **then continue**
5: **if** not $\text{Overlap}(e_1, e_2)$ **then continue**
6: $d \leftarrow \text{CalculateDistance}(e_1, e_2)$
7: **if** $d < \theta$ **then**
8: $\text{DedupAndRecord}(V, e_1, e_2)$
9: **return** V

bin are encapsulated into independent task units, allowing coarse-grained parallelism. Finally, rule-type identifiers are retained in the encapsulation, allowing task-level parallelism.

4.3 Distance-Rule Processing Paradigm

As described in Section 2.1, we can design a processing paradigm for distance rules by extracting their commonalities and isolating their differences, thereby constructing a unified pipeline structure on FPGAs to maximize hardware reuse and improve acceleration efficiency. Therefore, to better exploit the parallelism potential of DRC, we propose a hardware-friendly distance-rule processing paradigm outlined in Algorithm 1. The process begins with rule-specific pruning, which selects relevant candidate edge pairs for each rule (line 4). Next, projection overlap detection filters out pairs that could result in violations (line 5). Then, the Euclidean distance of the overlapping pairs is calculated and compared with the rule threshold to identify violations (lines 6–7). Finally, violations are recorded using a deduplication mechanism to avoid duplicate reports (line 8).

5 Hardware Acceleration Architecture

5.1 Architecture Overview

DRC exhibits three favorable properties for FPGA acceleration: locality, structural commonality across distance rules, and sparse violation outcomes. *AccDRC* leverages these properties to construct an FPGA acceleration framework composed of three tightly coordinated modules, as shown in Fig. 4(b).

Data Load. *AccDRC* leverages locality by packaging only candidate edges into compact task units and delivering them to the FPGA as contiguous memory blocks. This design ensures that each task can be processed entirely within a local working set, eliminating global data dependence and reducing memory irregularity.

Check Core. *AccDRC* exploits rule commonality by implementing a unified, reconfigurable distance-rule pipeline. A shared datapath serves all rule types, enabling high-throughput parallel checking without duplicating hardware for different rules.

Result Process. *AccDRC* takes advantage of sparse violation outcomes through a streaming, index-only writeback mechanism. By outputting only the violating edge pairs rather than a full comparison matrix, it dramatically reduces I/O bandwidth demands and enables fully overlapped dataflow execution.

Based on this architecture, *AccDRC* achieves a tightly coupled mapping from DRC algorithmic structure to hardware execution, enabling VLSI DRC to run on FPGA with high locality, high parallelism, and high bandwidth efficiency.

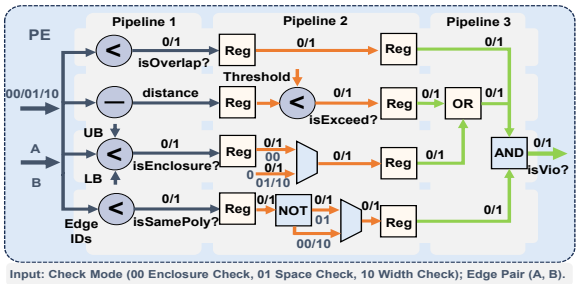


Figure 6: The PE of check core, implementing the unified distance-rule pipeline for space, width, and enclosure checks.

5.2 Reconfigurable Checker Array

Based on the unified distance-rule paradigm introduced in Section 4.3, the checker array in *AccDRC* adopts a shared datapath with lightweight control logic to differentiate rule types and enable dynamic run-time switching.

Checker Pipeline. As shown in Fig. 6, the array consists of multiple parallel PEs, each implemented as a three-stage pipeline. This pipeline directly maps algorithmic dependencies onto hardware timing, achieving a deterministic throughput of one edge pair per cycle. The PE array performs parallel comparisons and arithmetic operations on input edges to efficiently generate the violation flags. All rule types share the same underlying arithmetic units, while rule-specific differences are handled through low-overhead control paths, ensuring efficient resource utilization.

Rule Reconfiguration. The architecture supports run-time reconfiguration: using a two-level multiplexer and a 2-bit mode signal, it can rapidly switch among spacing, width, and enclosure modes. This run-time configurability requires neither resynthesis nor bitstream reload, significantly enhancing system flexibility and generality while avoiding the area overhead of conventional single-rule accelerators.

Overall, the checker array maps the distance-rule paradigm directly to a reconfigurable hardware structure, providing *AccDRC* with a high-throughput, general-purpose computation engine.

5.3 Multi-Level Parallelism

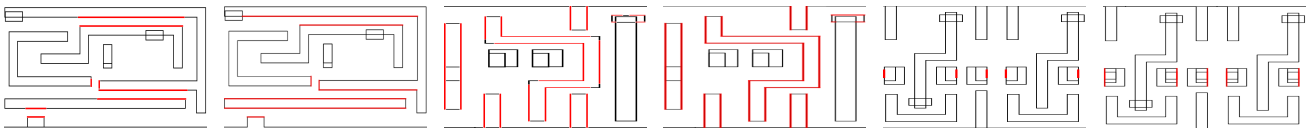
DRC exposes independence at the edge-pair, bin, and rule levels. *AccDRC* builds on this structure to realize coordinated parallelism across data, task, and rule-level dimensions.

Data Level. Each PE incorporates a multi-path comparison unit that evaluates several candidate edges in parallel on the unified pipeline, providing arithmetic-level parallelism without affecting pipeline timing. In addition, the shared datapath allows these parallel comparisons to reuse intermediate results such as projection bounds, further improving per-cycle computational efficiency.

Task Level. Multiple PEs run concurrently with bins as the scheduling unit. The bin partition matches the task-aware on-chip memory layout, ensuring each bin maps to an independent data block and a dedicated PE. With HBM multi-channel access and asynchronous double buffering, loading, computation, and writeback are overlapped, preventing bandwidth from throttling throughput.

Table 1: Runtime (s) comparison among KLayout, OpenDRC, and *AccDRC*.

Design	Space				Width				Enclosure			
	KLayout (Flat)	KLayout (Deep)	OpenDRC	*AccDRC*	KLayout (Flat)	KLayout (Deep)	OpenDRC	*AccDRC*	KLayout (Flat)	KLayout (Deep)	OpenDRC	*AccDRC*
aes	11.39	5.98	0.07	0.01	2.50	4.36	0.03	0.01	5.04	12.21	0.14	0.01
ethmac	45.16	26.71	0.43	0.03	10.78	13.08	0.64	0.03	18.85	58.54	0.52	0.04
jpeg	50.91	29.35	0.39	0.03	10.78	14.35	0.19	0.03	26.90	50.82	0.51	0.04
swerv	176.86	52.05	0.56	0.04	25.43	25.85	0.41	0.04	18.84	8.89	1.12	0.03
bp	362.79	144.04	2.08	0.08	58.63	68.09	0.76	0.09	52.62	20.00	2.86	0.05
Average	2584.98×	1071.03×	14.22×	1×	433.58×	522.11×	9.62×	1×	636.20×	867.97×	26.07×	1×



(a) (b) (c) (d) (e) (f)

Figure 7: Comparison of KLayout and *AccDRC* results for three checks on *aes*. (a) Space check by KLayout; (b) Space check by *AccDRC*; (c) Width check by KLayout; (d) Width check by *AccDRC*; (e) Enclosure check by KLayout; (f) Enclosure check by *AccDRC*. Red segments indicate violating edge pairs; minor visual differences arise from different segment-length conventions.

Rule Level. Run-time reconfigurable control enables adaptive multi-mode scheduling. The system can fix the mode to maximize throughput for single-rule workloads or time-slice among modes for mixed-rule scenarios, supporting multi-rule checking without increasing hardware area.

By coordinating parallelism across these three layers, *AccDRC* establishes a scalable execution architecture based on local computation, independent task dispatch, and dynamic mode switching, achieving high parallelism and stable high-throughput operation.

5.4 Result Compression and Writeback

Although the number of edge pairs is extremely large, actual violations are sparse. Returning all comparison results in a 0/1 format would generate an $O(N^2)$ sparse matrix whose size far exceeds PCIe bandwidth. To prevent I/O bottleneck, we employs a hardware-level sparse compression module that outputs only the indices of violating edge pairs.

This module operates in parallel with the checker array and consists of: (1) a priority encoder that extracts all "1" positions from the violation bitmap in real time, where each "1" indicates an edge pair identified as a violation; (2) an index-mapping unit that converts them into global edge-pair indices with bin and rule information; (3) an FIFO packer that assembles the index stream into PCIe-ready frames.

The entire process is fully streaming—no need to wait for all bins—and overlaps with the main pipeline. This mechanism reduces output data by over 90% and significantly lowers bandwidth usage. Because the module is decoupled from the critical path, it can scale to multiple output channels without affecting timing, further improving system throughput.

6 Experimental Results

6.1 Experimental Setup

All experiments are conducted on a Linux server with an 8-core Intel Xeon Gold 5218R processor (2.10 GHz) and 64 GB of memory. The system integrates an Xilinx Alveo U280 FPGA accelerator card, which features 8 GB of HBM2 high-bandwidth memory, 1.3 million logic cells, 2,160 blocks of Block RAM, and 960 blocks of UltraRAM. The accelerator cards connect to the host via PCIe Gen4 x8 interfaces, enabling high-throughput data transfer. The GPU adopts an NVDIA A100 accelerator.

To evaluate the efficiency of *AccDRC*, we compared its performance with two state-of-the-art tools: the CPU-based DRC tool KLayout, executed with 16 threads in both flat and deep (hierarchical) modes, and the GPU-based DRC tool OpenDRC [6]. Since OpenDRC has only released its sequential version, we re-implemented its parallel version based on the description in the original paper. Benchmark designs *swerv* and *bp* are synthesized with the FreePDK 45nm PDK [1], while *aes*, *ethmac*, and *jpeg* are synthesized with the ASAP 7 nm PDK [3], all using OpenROAD [12].

6.2 Performance Evaluation

Runtime Comparison with KLayout and OpenDRC. Table 1 presents the runtime comparisons of KLayout (flat/deep), OpenDRC, and *AccDRC*. For space check, *AccDRC* achieves speedups of 2584.98× over KLayout (flat), 1071.03× over KLayout (deep), and 14.22× over OpenDRC. For width check, the speedups are 433.58×, 522.11×, and 9.62×, respectively; for enclosure check, they are 636.20×, 867.97×, and 26.27×, respectively. Moreover, *AccDRC* exhibits minimal runtime variation across different design scales, whereas KLayout and OpenDRC show significant time increases as scale increases. These results indicate that *AccDRC* achieves more efficient performance on large-scale designs, maintaining stability and avoiding the scalability bottlenecks observed in existing tools.

Accuracy Comparison with KLayout. Fig. 7 shows visualization results of three checks on the `aes` design, obtained by KLayout (flat) and *AccDRC*, with red segments denoting violating edge pairs. Although some violations are recorded with different segment lengths, *AccDRC* yields the same results as KLayout, demonstrating its correctness in processing layout data on the CPU side and its accuracy in executing DRC functions on the FPGA side.

Table 2: Runtime (s) comparison among *AccCDRC*, *AccGDRC*, and *AccDRC*.

Design	Space			Width			Enclosure		
	AccCDRC	*AccGDRC*	*AccDRC*	*AccCDRC*	*AccGDRC*	*AccDRC*	*AccCDRC*	*AccGDRC*	*AccDRC*
aes	0.94	0.09	0.01	0.31	0.08	0.01	2.55	0.27	0.01
ethmac	4.01	0.34	0.03	3.96	0.57	0.03	9.50	0.79	0.04
jpeg	4.73	0.37	0.03	1.43	0.27	0.03	11.78	1.39	0.04
swerv	5.38	0.52	0.04	1.96	0.54	0.04	3.41	0.27	0.03
bp	10.60	1.12	0.08	4.34	1.16	0.09	8.03	0.60	0.05
Average	120.98×	11.05×	1×	65.13×	13.39×	1×	198.06×	18.92×	1×

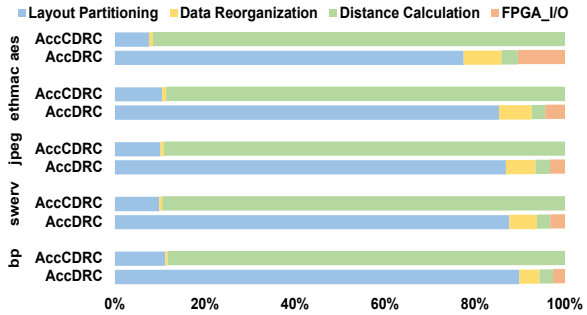


Figure 8: Runtime breakdown of *AccDRC* and *AccCDRC* for space checks, showing the contributions of layout partitioning, data reorganization, DRC Check, and FPGA I/O.

Runtime Comparison under Different Parallel Acceleration Architectures. Table 2 presents the runtime comparisons of our framework executed on three parallel acceleration architectures: *AccDRC*, a CPU-based implementation with 16 threads; AccGDRC, a GPU-based implementation; and *AccDRC*, an FPGA-based hardware implementation. In space check, *AccDRC* achieves speedups of 120.98× over *AccCDRC* and 11.05× over AccGDRC. In width check, the speedups are 65.13× and 13.39×. In enclosure check, they are 198.06× and 18.92×, respectively. The comparison confirms that the FPGA-based architecture offers the most significant acceleration among the three platforms, validating its effectiveness as a high-performance solution for DRC acceleration.

6.3 Runtime Analysis

Fig. 8 shows the runtime breakdown of *AccDRC* and *AccDRC* during space checks on five designs. In *AccDRC*, distance calculation accounts for more than 80% of the runtime and constitutes the primary bottleneck. After FPGA acceleration, *AccDRC* significantly reduces the time spent on distance calculation, with layout partitioning becoming the new major time-consuming step, which can be considered a target for future optimization. Data preparation and FPGA I/O account for a smaller proportion of the total runtime, but in large-scale designs, data compression remains important to further reduce transmission overhead.

6.4 Hardware Resource Utilization

Table 3 summarizes the hardware resource utilization of *AccDRC*. The utilization rates for LUT, BRAM, and URAM are 50.7%, 75.1%, and 76.25%, respectively, indicating that *AccDRC* carries a high workload in logic computation and on-chip storage, primarily for processing edge data and intermediate caches. Clock and I/O resource consumption is relatively low, with only a single PCIe lane sufficient to meet current data-transfer needs. Overall, *AccDRC* maintains reasonably balanced resource utilization while sustaining parallel computing capabilities, demonstrating its scalability and deployment flexibility. The on-chip power consumption of the *AccDRC* implementation, reported by Vivado under the resource utilization of 599K LUTs, 242K FFs, 1622 BRAMs, and 732 URAMs on the Alveo U280, is 43.27 W.

Table 3: HARDWARE RESOURCE UTILIZATION.

Resource	Utilization	Available	Percent (%)
LUT	599354	1182240	50.70
LUTRAM	157596	591840	26.63
FF	241891	2364480	10.23
BRAM	1622	2160	75.10
URAM	732	960	76.25
IO	4	832	0.48
BUFG	24	1800	1.33
MMCM	2	12	16.67
PCIe	1	6	16.67

7 Conclusion

This paper proposes *AccDRC*, an FPGA-accelerated DRC with cell-aware partitioning. On the software, we organizes layouts into balanced, locality-preserving task units and prepares hardware-friendly edge data through pruning identifiers and task encapsulation. On the hardware, we implements an acceleration architecture consisting of a locality-preserving data-loading module, a unified and reconfigurable check core, and a sparse result writeback module. Experimental results show that *AccDRC* delivers substantial speedup over both CPU- and GPU-based DRC tools while maintaining checking accuracy.

Future work will extend this FPGA acceleration architecture to more DRC rules by generalizing the unified processing paradigm beyond distance rules.

Acknowledgments

This work was partially supported by the National Natural Science Foundation of China under Grant No. 62372109, and the Fujian Natural Science Funds under Grant No. 2023J06017.

References

[1] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, Geraldo Pradipta, Sherief Reda, Mehdi Saligane, Sachin S. Sapatnekar, Carl Sechen, Mohamed Shalan, William Swartz, Lutong Wang, Zhehong Wang, Mingyu Woo, and Bangqi Xu. 2019. Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project. In *Proceedings of the 56th ACM/IEEE Design Automation Conference (DAC)*.

[2] E.C. Carlson and R.A. Rutenbar. 1987. A Scanline Data Structure Processor for VLSI Geometry Checking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 6, 5 (1987), 780–794.

[3] Lawrence T. Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectronics Journal* 53 (2016), 105–115. doi:10.1016/j.mejo.2016.04.006

[4] Zhuolun He, Yuzhe Ma, and Bei Yu. 2022. X-Check: GPU-Accelerated Design Rule Checking via Parallel Sweepline Algorithms. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.

[5] Zhuolun He and Bei Yu. 2023. Invited Paper: Heterogeneous Acceleration for Design Rule Checking. In *Proceedings of the IEEE/ACM International Conference on Computer Aided Design (ICCAD)*.

[6] Zhuolun He, Yihang Zuo, Jiaxi Jiang, Haisheng Zheng, Yuzhe Ma, and Bei Yu. 2023. OpenDRC: An Efficient Open-Source Design Rule Checking Engine with Hierarchical GPU Acceleration. In *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*.

[7] Jiaxi Jiang, Lancheng Zou, Wenqian Zhao, Zhuolun He, Tinghuan Chen, and Bei Yu. 2024. PDRC: Package Design Rule Checking via GPU-Accelerated Geometric Intersection Algorithms for Non-Manhattan Geometry. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*.

[8] R. Kane and S. Sahni. 1984. A Systolic Design Rule Checker. In *Proceedings of the 21st ACM/IEEE Design Automation Conference (DAC)*.

[9] Zhen Luo, Margaret Martonosi, and Pranav Ashar. 2000. An Edge-endpoint-based Configurable Hardware Architecture for VLSI Layout Design Rule Checking. *VLSI Design* 10, 3 (2000), 249–263.

[10] S. K. Nandy. 1994. Geometric Design Rule Check of VLSI Layouts in Distributed Computing Environment. *VLSI Design* 1, 2 (1994), 054126.

[11] A.P.V. Pais, M.L. Anido, and C.E.T. Oliveira. 2001. Developing a distributed architecture for design rule checking. In *Proceedings of the 44th IEEE 2001 Midwest Symposium on Circuits and Systems (MWSCAS)*.

[12] James E. Stine, Ivan Castellanos, Michael Wood, Jeff Henson, Fred Love, W. Rhett Davis, Paul D. Franzon, Michael Bucher, Sunil Basavarajaiah, Julie Oh, and Ravi Jenkal. 2007. FreePDK: An Open-Source Variation-Aware Design Kit. In *Proceedings of the IEEE International Conference on Microelectronic Systems Education (MSE)*.

[13] Yifei Zhou, Zijian Wang, and Chao Wang. 2024. E2E-Check: End to End GPU-Accelerated Design Rule Checking with Novel Mask Boolean Algorithms. In *Proceedings of the 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*.